

Cola de Prova - Programação Orientada a Objetos

Java + Python | índice rápido com páginas | baseado nos PDFs anexados e no exercício da imagem

Assunto	p.	Assunto	p.
1. Visão geral da disciplina e paradigmas	2	9. Sintaxe Essencial em Java	9
2. Fundamentos de POO: classes, objetos, estado e operações	2	Modelo completo de programa Java	9
Classes e objetos	2	Herança, interface e classe abstrata em Java	9
3. Java: conteúdo essencial das aulas	3	Coleções e estruturas úteis em Java	10
Estrutura de uma classe Java	3	10. Sintaxe Essencial em Python	10
Exceções, entrada e saída	4	Modelo completo de programa Python	10
Arrays, matrizes e String	4	Herança, propriedades e métodos especiais em Python	10
4. Python: elementos básicos, comandos e classes	4	Coleções e laços em Python	11
Strings, listas, matrizes, tuplas e dicionários	5	11. Tutoriais de Resolução de Problemas Comuns	11
Entrada, saída, arquivos, exceções e match	5	Criar uma classe a partir de um enunciado	11
Classes em Python	6	Resolver questões com herança	11
5. Herança em Java e Python	6	Resolver questões com polimorfismo	11
Java: extends, super e sobrescrita	6	Interfaces e classes abstratas	12
Classes abstratas	6	Resolver questões com listas de objetos	12
Herança múltipla e MRO em Python	7	Resolver questões com menus	12
6. Polimorfismo, sobrescrita e sobrecarga	7	12. Soluções dos Problemas da Imagem Anexada	13
Polimorfismo por subtipagem	7	Modelagem das classes	13
Duck typing em Python	7	Solução completa em Java	13
7. Interfaces e tipos genéricos	8	Solução completa em Python	15
Interfaces	8	13. Comparação Java vs Python	16
Genéricos	8	14. Checklist de Prova	16
8. Estudo de caso: jogo de Bozó	8	Erros comuns em Java	16
Regras do Bozó	8	Erros comuns em Python	16
Classes do Bozó	8	15. Fontes usadas	17
Fluxo do main do Bozó	9		

Legenda: os tópicos em azul escuro são seções principais; os recuados são atalhos de consulta. Códigos completos estão nas seções de sintaxe, tutoriais e soluções da imagem.

1. Visão geral da disciplina e paradigmas

A disciplina trabalha Programação Orientada a Objetos como um paradigma imperativo que modela sistemas por objetos. Um paradigma é um estilo de organizar programas e raciocinar sobre a solução: procedural, orientado a objetos, funcional e lógico são exemplos comuns.

Paradigma	Ideia central	Quando aparece na prova
Procedural	Programa organizado em procedimentos/funções e sequência de comandos.	Comparar com POO; lembrar de C/Pascal/Basic.
Orientado a objetos	Sistema modelado por objetos, que são instâncias de classes e interagem por métodos.	Classes, objetos, estado, comportamento, encapsulamento, herança, polimorfismo.
Funcional	Ênfase em funções, imutabilidade e composição de funções.	Pode aparecer conceitualmente na comparação de paradigmas.
Lógico	Problema descrito por fatos, regras e inferência lógica.	Exemplos: Prolog, Datalog; comparação de abordagem.

- Categorias: paradigmas imperativos dizem como executar passos; declarativos dizem o que deve ser obtido.
- Em POO, os quatro pilares mais cobrados são encapsulamento, abstração, herança e polimorfismo.
- Benefícios: reutilização de código, modelagem do mundo real e manutenção simplificada. Desafios: abstração adequada e aumento de complexidade em projetos grandes.
- Conteúdo da disciplina: Java e Python, classes, subclasses, herança, polimorfismo, interfaces, genéricos, exercícios e implementação prática.

2. Fundamentos de POO: classes, objetos, estado e operações

Classes e objetos

Classe é o molde. Objeto é a instância concreta criada a partir desse molde. Uma classe define atributos, que representam o estado, e métodos, que representam operações/comportamentos.

Termo	Significado	Como reconhecer no enunciado
Classe	Modelo/forma de criar objetos.	Substantivo principal: Pessoa, Cadeira, Elevador, Dado.
Objeto	Instância individual de uma classe.	Um item concreto: cadeira1, pessoaAna, dado6.
Atributo	Variável guardada no objeto.	Características: nome, peso, altura, posição, ocupado.
Método	Operação que o objeto sabe executar.	Verbos: sentar, levantar, rolar, remover, calcular.
Construtor	Método especial que inicializa o objeto.	Valores iniciais exigidos no momento de criar o objeto.

Obs.: Nos slides, a cadeira é usada para separar estado e operações: estado = posição e ocupado; operações = sentar, levantar e virar. Essa separação é a base para transformar enunciados em classes.

Exemplo central: classe Cadeira

```
public class Cadeira {
    private String posicao;
    private boolean ocupado;

    public Cadeira() {
        this.posicao = "Em pé";
        this.ocupado = false;
    }

    public Cadeira(String posicao, boolean ocupado) {
        this.posicao = posicao;
        this.ocupado = ocupado;
    }

    public void sentar() {
        if (!ocupado && posicao.equals("Em pé")) {
            ocupado = true;
        }
    }

    public void levantar() {
        ocupado = false;
    }

    public void virar() {
        if (posicao.equals("Em pé")) {
            posicao = "Invertida";
            ocupado = false;
        } else {
            posicao = "Em pé";
        }
    }

    public String getPosicao() { return posicao; }
    public boolean isOcupado() { return ocupado; }

    @Override
    public String toString() {
        return "Cadeira{posicao='" + posicao + "', ocupado=" + ocupado + "}";
    }
}
```

- Atributos estão privados para proteger o estado interno.
- Métodos públicos são a interface de uso da classe.
- Construtor sem parâmetros cria estado padrão; construtor com parâmetros permite estado inicial específico.
- this.posicao acessa o atributo do objeto; posicao sozinho acessa o parâmetro/local quando há conflito de nomes.

Construtores, atributos e métodos

- Construtor em Java tem o mesmo nome da classe e não possui tipo de retorno. Pode haver sobrecarga de construtores.
- Em Python, o construtor é `__init__(self, ...)`. Python não permite dois métodos com o mesmo nome; usa parâmetros padrão para simular múltiplas formas de construir.
- Atributos devem ser inicializados em um estado válido. Em Python, normalmente são definidos dentro do `__init__`.
- Métodos devem manter as invariantes da classe. Ex.: uma cadeira invertida não deve continuar ocupada.

Encapsulamento e ocultação de informação

Encapsular é esconder detalhes internos e expor apenas operações seguras. Em Java, usa-se `private` nos atributos e `public` nos métodos necessários. Em Python, usa-se convenção de sublinhado e, quando necessário, `@property`.

Nível	Java	Python
Público	<code>public</code>	sem sublinhado: nome
Privado/interno	<code>private</code>	convenção: <code>_nome</code> ; name mangling: <code>__nome</code>
Protegido	<code>protected</code>	convenção: <code>_nome</code>
Acesso controlado	getters/setters	<code>@property</code> e <code>setter</code>

Atenção: Atributo privado não significa que nunca muda; significa que deve mudar por métodos da classe, permitindo validação. Em prova, quase sempre use `private` em Java e crie getters/setters quando o enunciado pedir consultar ou alterar.

Composição, associação e agregação

Composição ocorre quando um objeto é formado por outros objetos e normalmente controla o ciclo de vida deles. Nos slides, um centro de eventos ou uma sala pode possuir várias cadeiras: a classe gerenciadora tem um vetor/lista de Cadeira.

Relação	Ideia	Exemplo de código/modelagem
Associação	Um objeto conhece/usa outro.	ControleRemoto controla uma Televisao.
Agregação	Tem-um fraco: parte pode existir sem o todo.	Agenda guarda Pessoas; pessoa existe fora da agenda.
Composição	Tem-um forte: todo cria/gerencia as partes.	Sala cria e mantém array de Cadeiras.
Herança	É-um: subclasse é tipo especializado.	Gato é Animal; Cachorro é Animal.

Membros estáticos

Um membro estático pertence à classe, não a cada objeto. Nos slides, `conta_cadeiras` é exemplo: cada cadeira tem posição e ocupado, mas a contagem total é da classe Cadeira.

```
class Cadeira {
    private static int contaCadeiras = 0;
    public Cadeira() { contaCadeiras++; }
    public static int getContaCadeiras() { return contaCadeiras; }
}
```

Exercícios de modelagem citados nas aulas

- Pessoa: atributos nome, data de nascimento, peso, altura; métodos getters/setters e calcular idade.
- Agenda: armazena até 100 pessoas; insere, remove, busca por nome e busca por posição.
- Elevador: andar atual, total de andares, capacidade e pessoas presentes; subir, descer, entrar, sair.
- Televisão e ControleRemoto: controle aumenta/diminui volume, muda canal e consulta estado da TV.
- Tabuleiro de números: representar estado, movimentos e método que testa se é solução.
- Dado: número de lados configurável, rolar e consultar último lado.

3. Java: conteúdo essencial das aulas

Estrutura de uma classe Java

- Uma classe pública `public class Cadeira` deve ser salva em `Cadeira.java`.
- Cada membro tem seu próprio controle de acesso: atributos, métodos e construtores podem ser `public`, `private` ou `protected`.
- Java tem verificação estática de tipos: todos os membros e variáveis têm tipo conhecido em compilação.
- O programa normalmente começa em `public static void main(String[] args)`.

```
public class Main {
    public static void main(String[] args) {
        Cadeira c1 = new Cadeira();
        Cadeira c2 = new Cadeira("Invertida", false);
        c1.sentar();
        c2.virar();
        System.out.println(c1.getPosicao());
        System.out.println(c2.getPosicao());
    }
}
```

Tipos, variáveis, escopo e casting

Tipo	Tamanho típico nos slides	Uso
<code>int</code>	4 bytes	inteiros comuns
<code>long</code>	8 bytes	inteiros maiores
<code>float</code>	4 bytes	ponto flutuante menor precisão
<code>double</code>	8 bytes	ponto flutuante padrão
<code>char</code>	2 bytes	caractere Unicode
<code>boolean</code>	depende da JVM	<code>true/false</code>
<code>String</code>	objeto	texto; comparar com <code>equals</code> , não com <code>==</code>

- Escopo: variável declarada dentro de `if`, `for` ou método só existe naquele bloco.

- Parâmetros são variáveis locais do método.
- Casting é obrigatório quando pode haver perda de informação: `double x = 3.2; int n = (int) x;`
- Conversões automáticas normalmente só ocorrem quando não há perda relevante. como `int` para `double`.

```
int a = 10;
double b = a;      // ok: int -> double
int c = (int) b;    // casting explícito
String s = "abc";
int k = s.length();
String parte = s.substring(0, 2);
boolean igual = s.equals("abc");
```

Operadores e comandos de controle

Categoria	Operadores/comandos
Aritméticos	+, -, *, /, %, ++, --
Comparação	==, !=, <, <=, >, >=
Lógicos	!, &&,
Bit/shift	~, &, , ^, <<, >>, >>>
Condição	if/else, switch
Repetição	for, enhanced for, while, do while
Controle de laço	break, continue; pode haver rótulo para quebrar laço externo

```
if (x > 0) {
    System.out.println("positivo");
} else {
    System.out.println("zero ou negativo");
}

switch (op) {
    case 'u': t.up(); break;
    case 'd': t.down(); break;
    default: System.out.println("opção inválida");
}

for (int i = 0; i < 10; i++) { }
for (int valor : vetor) { }
while (i < 10) { i++; }
do { i++; } while (i < 10);
```

Exceções, entrada e saída

- Exceção é um mecanismo para sinalizar erros sem usar códigos de retorno como em C.
- try contém o código que pode falhar; catch trata a exceção; finally executa ao final, falhando ou não.
- Em leitura de arquivo, `FileNotFoundException` e `IOException` são comuns.
- Para entrada simples em prova. `Scanner` é mais curto. `BufferedReader` aparece nos slides para leitura de texto.

```
try {
    Scanner sc = new Scanner(System.in);
    String nome = sc.nextLine();
    int idade = Integer.parseInt(sc.nextLine());
    System.out.println(nome + " " + idade);
    sc.close();
} catch (NumberFormatException e) {
    System.out.println("Idade inválida");
} catch (Exception e) {
    System.out.println("Erro geral: " + e.getMessage());
}
```

Arrays, matrizes e String

```
int[] table = new int[16];
for (int i = 0; i < table.length; i++) table[i] = i;

int[][] matriz = new int[4][4];
int k = 0;
for (int i = 0; i < matriz.length; i++) {
    for (int j = 0; j < matriz[i].length; j++) {
        matriz[i][j] = k++;
    }
}

String msg = "Digite uma opção (U/D/L/R) ou Q para sair: ";
int tam = msg.length();
String sub = msg.substring(4, 10);
boolean ok = msg.equals("texto"); // nunca use == para comparar conteúdo de String
```

4. Python: elementos básicos, comandos e classes

Interpretador, script e indentação

- Python pode ser usado no interpretador interativo ou executando um arquivo `.py`.
- É linguagem de script: o primeiro comando fora de classes/funções inicia a execução.
- A indentação define blocos; não há chaves obrigatórias como em Java.
- Um arquivo pode conter várias classes e código principal. Use `if __name__ == "__main__"` para separar teste/execução.

Números, operadores, variáveis e conversões

- `int` representa inteiros; `float` representa ponto flutuante. 3 é `int`, 3.0 é `float`.
- A variável não tem tipo fixo; o valor armazenado nela tem tipo.
- Operadores: +, -, *, /, **; divisão inteira //; resto %.
- Precedência destacada nos slides: potência, operadores unários, multiplicação/divisão, soma/subtração.

- Conversões: `int("3").float("4.6").str(3).`

```
x = 10
x = x ** 2
y = 3.0 + float("4.6")
texto = str(3) + "abc"
q = 7 // 2 # 3
r = 7 % 2 # 1
```

Strings, listas, matrizes, tuplas e dicionários

```
nome = "Python"
mensagem = "Olá, mundo!"
c = nome[2]
d = mensagem[3:8]
k = len(nome)
```

```
nomes = ["Alice", "Bob", "Carol", "Dave"]
ultimo = nomes[-1]
nomes.append("Eva")
nomes.insert(0, "Zeca")
nomes.remove("Bob")
sub = nomes[1:3]
```

```
matriz = [[0,1,2], [3,4,5], [6,7,8]]
valor = matriz[1][1]
linha = matriz[1]
```

```
coordenada = (10, 20) # tupla: imutável
idades = {"Ana": 20, "Bruno": 22} # dicionário: chave -> valor
```

Obs.: Listas podem misturar tipos, mas em questões de POO prefira listas homogêneas, como lista de Pessoa, lista de Cadeira ou lista de Par. Tuplas são como listas que não devem ser modificadas.

Entrada, saída, arquivos, exceções e match

```
x0 = float(input("Forneça o valor inicial: "))
print(x0)
print("Solução: {:.7f}".format(x0))

try:
    with open("Entrada.txt", "r", encoding="utf-8") as f:
        for linha in f:
            print(linha.strip())
except OSError:
    print("Erro ao abrir arquivo")

op = input("Opção: ")
match op:
    case "u": print("up")
    case "d": print("down")
    case _: print("opção inválida")
```

Funções e passagem de parâmetros

Nos slides, a passagem de parâmetros é explicada como passagem por valor de referência de objeto: a função recebe uma referência para o mesmo objeto. Se o objeto é mutável, como uma lista, alterações internas aparecem fora da função.

```
def modificar_lista(lista):
    lista.append(4)

minha_lista = [1, 2, 3]
modificar_lista(minha_lista)
print(minha_lista) # [1, 2, 3, 4]

# parâmetros opcionais substituem a sobrecarga de métodos de Java
def altura(no=0):
    return no + 1

altura()
altura(16)
altura(no=16)
```

Classes em Python

```
class Cadeira:
    def __init__(self, p="Em pé", oc=False):
        self.posicao = p
        self.ocupado = oc

    def sentar(self):
        if not self.ocupado and self.posicao == "Em pé":
            self.ocupado = True

    def levantar(self):
        self.ocupado = False

    def virar(self):
        if self.posicao == "Em pé":
            self.posicao = "Invertida"
            self.ocupado = False
        else:
            self.posicao = "Em pé"

    def get_posicao(self):
        return self.posicao

if __name__ == "__main__":
    c1 = Cadeira()
    c2 = Cadeira("Invertida", False)
    c1.sentar()
    c2.virar()
    print(c1.get_posicao())
    print(c2.get_posicao())
```

5. Herança em Java e Python

Conceito e vocabulário

- Herança permite que uma classe herde atributos e métodos de outra.
- Classe pai/superclasse: mais geral. Classe filha/subclasse: mais específica.
- Vantagem principal: reutilização de código e criação de hierarquias.
- Subclasse pode adicionar novos comportamentos ou substituir comportamentos existentes.
- Relação correta de herança é "é um": Gato é Animal; Cachorro é Animal. Agenda não é Pessoa, então não é herança.

Java: extends, super e sobrescrita

```
abstract class Animal {
    private String nome;

    public Animal(String nome) { this.nome = nome; }
    public String getNome() { return nome; }
    public abstract void fazerSom();

    @Override
    public String toString() { return nome; }
}

class Gato extends Animal {
    public Gato(String nome) { super(nome); }

    @Override
    public void fazerSom() { System.out.println("O gato mia"); }
}

class Cachorro extends Animal {
    public Cachorro(String nome) { super(nome); }

    @Override
    public void fazerSom() { System.out.println("O cachorro late"); }

    public void levantarPata() { System.out.println("Fiz xixi"); }
}

public class Main {
    static void darBanho(Animal a) { a.fazerSom(); } // polimorfismo

    public static void main(String[] args) {
        Animal a1 = new Gato("Mimi");
        Animal a2 = new Cachorro("Rex");
        darBanho(a1);

        darBanho(a2);
        // a2.levantarPata(); // erro: o tipo declarado é Animal
        if (a2 instanceof Cachorro) ((Cachorro) a2).levantarPata();
    }
}
```

Atenção: No exemplo dos slides, `Animal a = new Cachorro("Rex");` permite `a.fazerSom()`, mas não permite `a.levantarPata()`, porque o tipo declarado é `Animal`. O método chamado dinamicamente é o da subclasse; mas o conjunto de métodos acessíveis é definido pelo tipo da referência.

Classes abstratas

Uma classe abstrata representa uma ideia incompleta. Nos slides aparece a ideia de que "ninguém adota um animal" genérico: adota-se um Gato ou Cachorro. Logo, `Animal` pode ser abstrata.

- Java: abstract class pode ter atributos, construtores, métodos concretos e métodos abstract.
- Não é possível instanciar classe abstrata diretamente.
- Subclasses concretas devem implementar os métodos abstratos.
- Python: use `abc.ABC` e `@abstractmethod` quando quiser impor implementação.

```

from abc import ABC, abstractmethod

class Animal(ABC):
    def __init__(self, nome):
        self.nome = nome

    @abstractmethod
    def fazer_som(self):
        pass

class Gato(Animal):
    def fazer_som(self):
        print("O gato mia")

class Cachorro(Animal):
    def fazer_som(self):
        print("O cachorro late")

    def levantar_pata(self):
        print("Fiz xixi")

def dar_banho(animal):
    animal.fazer_som() # duck typing: basta ter esse método

if __name__ == "__main__":
    animais = [Gato("Mimi"), Cachorro("Rex")]
    for a in animais:
        dar_banho(a)

```

Herança múltipla e MRO em Python

Python aceita herança múltipla. Quando um método é chamado, a ordem de procura é a MRO - Method Resolution Order. Ela pode ser consultada com `Classe.mro()`.

```

class A:
    def m1(self): print("A.m1")
class B(A):
    def m1(self): print("B.m1")
class C(A):
    def m1(self): print("C.m1")
class D(B, C):
    pass

d = D()
d.m1() # procura em D, B, C, A, object
print(D.mro())

```

Atenção: O problema de construtores em herança múltipla ocorre porque classes diferentes podem exigir parâmetros diferentes. Em projetos simples de prova, evite herança múltipla salvo quando o enunciado pedir claramente.

6. Polimorfismo, sobrescrita e sobrecarga

Polimorfismo por subtipagem

Polimorfismo permite tratar vários tipos concretos de maneira homogênea por meio de um tipo mais abstrato. Em Java, uma variável do tipo `Animal` pode referenciar objetos `Gato` e `Cachorro`; em Python, basta que o objeto tenha os métodos usados.

- Chamada dinâmica: se a referência é `Animal`, mas o objeto real é `Cachorro`, `fazerSom` executa a versão de `Cachorro`.
- Métodos exclusivos da subclasse não ficam acessíveis por uma referência da superclasse sem cast.
- Polimorfismo aparece em parâmetros: `void darBanho(Animal an)` aceita `Gato` e `Cachorro`.
- Também aparece em vetores/listas: `Animal[]` ou `ArrayList<Animal>` pode guardar diferentes subclasses.

```

Animal[] animais = new Animal[3];
animais[0] = new Gato("Mimi");
animais[1] = new Cachorro("Rex");
animais[2] = new Cachorro("Toto");

for (Animal a : animais) {
    a.fazerSom(); // cada objeto executa sua própria versão
}

```

Duck typing em Python

Duck typing: se parece um pato, nada como pato e gralha como pato, trate como pato. Em Python, a herança formal é menos necessária para polimorfismo: o importante é o objeto ter os métodos usados.

```

class AnimalMisterioso:
    def fazer_som(self): print("som misterioso")

def tocar_som(obj):
    obj.fazer_som()

tocar_som(Gato("Mimi"))
tocar_som(AnimalMisterioso())

```

Sobrescrita vs sobrecarga

Conceito	Java	Python
Sobrescrita	Subclasse redefine método herdado. Use <code>@Override</code> .	Subclasse define método com mesmo nome; substitui o herdado.
Sobrecarga	Mesmo nome, parâmetros diferentes: <code>altura()</code> e <code>altura(int no)</code> .	Não existe do mesmo modo; último def com mesmo nome substitui o anterior.
Alternativa em Python	Não se aplica.	Use parâmetros opcionais/nomeados: <code>def altura(no=0)</code> .

7. Interfaces e tipos genéricos

Interfaces

- Interface é um template/contrato: define o que uma classe deve fazer, mas não como faz.
- Classes implementam interfaces com implements em Java.
- Uma classe pode implementar várias interfaces, o que resolve parte do problema de tratar classes de hierarquias diferentes como iguais.
- Em termos de polimorfismo, uma interface pode funcionar como tipo abstrato: uma variável do tipo da interface referencia qualquer objeto que a implemente.
- Interfaces Java podem declarar métodos abstratos e constantes public static final.

```
import java.util.*;

class Animal implements Comparable<Animal> {
    private String nome;
    public Animal(String nome) { this.nome = nome; }
    public String getNome() { return nome; }

    @Override
    public int compareTo(Animal outro) {
        return this.nome.compareTo(outro.nome);
    }

    @Override
    public String toString() { return nome; }
}

public class Main {
    public static void main(String[] args) {
        Animal[] todos = { new Animal("Jeremias"), new Animal("Kelle"), new Animal("Tadeu") };
        Arrays.sort(todos); // só funciona porque Animal implementa Comparable<Animal>
        System.out.println(Arrays.toString(todos));
    }
}
```

Atenção: Arrays.sort em um array de objetos pode gerar ClassCastException se os objetos não implementarem Comparable ou se nenhum Comparator for fornecido. Essa é a motivação do exemplo de Comparable nos slides.

Genéricos

Tipos genéricos permitem escrever classes, interfaces e métodos parametrizados por tipo. Em vez de criar uma PilhaDeInteiros e uma PilhaDeStrings, cria-se Pilha. O tipo concreto é definido na instanciação.

```
class Caixa<T> {
    private T valor;
    public Caixa(T valor) { this.valor = valor; }
    public T getValor() { return valor; }
}

Caixa<Integer> c1 = new Caixa<>(10);
Caixa<String> c2 = new Caixa<>("abc");
```

- Vantagem: segurança de tipos em tempo de compilação.
- Evita casts manuais e erros como colocar String em lista de Integer.
- Em listas ligadas, nós e listas normalmente usam o mesmo parâmetro T.
- No exercício da imagem, Par precisa de dois tipos genéricos, pois os dados podem ter tipos diferentes.

8. Estudo de caso: jogo de Bozó

Regras do Bozó

- Bozó é um jogo de dados; cada jogador tenta preencher 10 posições de um placar.
- Em cada rodada, o jogador lança 5 dados. Depois pode escolher de 1 a 5 dados para lançar novamente, até duas vezes.
- Ao final da rodada, escolhe uma posição ainda vazia do placar. A pontuação depende da posição e dos dados finais.
- Posições externas #1 a #6: soma dos dados que mostram o número da posição. Ex.: quatro dados com valor 3 na posição #3 valem 12.
- Posições internas: full hand vale 15, sequência 1-5 ou 2-6 vale 20, quadra vale 30, quina vale 40. Se não formar a configuração, marca zero.

Classes do Bozó

Classe	Responsabilidade
Dado	Simula um dado com número de lados configurável; usa Random; métodos getLado(), rolar() e toString().
RolaDados	Gerencia um conjunto de dados; rola todos ou apenas dados escolhidos; exhibe resultados.
Placar	Representa o placar; aloca combinações de dados às posições; mantém escore do jogador.
Bozo	Classe inicial com main; executa 10 rodadas e coordena interação com usuário.

```
class Dado {
    private int lados;
    private int lado;
    private java.util.Random random = new java.util.Random();

    public Dado() { this(6); }
    public Dado(int lados) { this.lados = lados; rolar(); }
    public int getLado() { return lado; }
    public int rolar() { lado = random.nextInt(lados) + 1; return lado; }
    public String toString() { return String.valueOf(lado); }
}

// Ideia de Placar: vetor de 10 posições, -1 se vazio.
// Posições 1..6: soma dos dados iguais à posição. Internas: full, sequência, quadra, quina.
```

Fluxo do main do Bozó

- Criar RolaDados e Placar.
- Repetir 10 vezes: esperar ENTER, rolar todos os dados e exibir.
- Pedir quais dados serão rolados novamente e rolar pela segunda vez.
- Pedir novamente os dados a rerrolar e rolar pela terceira vez.
- Mostrar placar atual, pedir posição, atualizar placar.
- No fim, mostrar pontuação total.

9. Sintaxe Essencial em Java

Modelo completo de programa Java

```
import java.util.*;
import java.io.*;

public class Exemplo {
    private String nome;           // atributo de instância
    private final int id;          // constante depois de inicializada
    private static int total = 0;  // atributo da classe, comum a todos os objetos

    public Exemplo(String nome, int id) { // construtor: mesmo nome da classe, sem retorno
        this.nome = nome;               // this diferencia atributo de parâmetro
        this.id = id;
        total++;
    }

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public static int getTotal() { return total; }

    public void imprimir() { System.out.println(nome + " #" + id); }
    public void imprimir(String prefixo) { System.out.println(prefixo + nome); } // sobrecarga

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Exemplo> lista = new ArrayList<>();
        HashMap<String, Exemplo> mapa = new HashMap<>();

        for (int i = 0; i < 3; i++) {
            String nome = sc.nextLine();
            Exemplo e = new Exemplo(nome, i);
            lista.add(e);
            mapa.put(nome, e);
        }

        for (Exemplo e : lista) e.imprimir();

        try {
            FileInputStream fis = new FileInputStream("entrada.txt");
            fis.close();
        } catch (IOException e) {
            System.out.println("Erro de E/S: " + e.getMessage());
        } finally {
            sc.close();
        }
    }
}
```

Herança, interface e classe abstrata em Java

```
interface Desenhavel {
    void desenhar();
}

abstract class Forma implements Desenhavel {
    private String cor;
    public Forma(String cor) { this.cor = cor; }
    public String getCor() { return cor; }
    public abstract double area();
}

class Circulo extends Forma {
    private double raio;
    public Circulo(String cor, double raio) {
        super(cor);
        this.raio = raio;
    }
    @Override public double area() { return Math.PI * raio * raio; }
    @Override public void desenhar() { System.out.println("círculo " + getCor()); }
}
```

Coleções e estruturas úteis em Java

```
int[] v = new int[10];
int[][] m = new int[3][4];

ArrayList<String> nomes = new ArrayList<>();
nomes.add("Ana");
nomes.get(0);
nomes.remove("Ana");
nomes.size();

HashMap<String, Integer> idades = new HashMap<>();
idades.put("Ana", 20);
int idade = idades.get("Ana");
boolean existe = idades.containsKey("Ana");

for (String nome : nomes) {
    System.out.println(nome);
}
```

10. Sintaxe Essencial em Python

Modelo completo de programa Python

```
class Exemplo:
    total = 0 # atributo da classe

    def __init__(self, nome, id=0):
        self.nome = nome # atributo do objeto
        self._id = id # convenção: uso interno/protegido
        Exemplo.total += 1

    @property
    def id(self):
        return self._id

    def imprimir(self, prefixo=""):
        print(f"{prefixo}{self.nome} #{self.id}")

if __name__ == "__main__":
    objetos = []
    mapa = {}

    for i in range(3):
        nome = input("Nome: ")
        e = Exemplo(nome, i)
        objetos.append(e)
        mapa[nome] = e

    for e in objetos:
        e.imprimir("Objeto: ")

    try:
        with open("entrada.txt", "r", encoding="utf-8") as f:
            conteudo = f.read()
    except OSError as erro:
        print("Erro de E/S:", erro)
```

Herança, propriedades e métodos especiais em Python

```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self._idade = idade

    @property
    def idade(self):
        return self._idade

    @idade.setter
    def idade(self, valor):
        if valor >= 0:
            self._idade = valor

    def __str__(self):
        return f"{self.nome} ({self.idade})"

class Aluno(Pessoa):
    def __init__(self, nome, idade, curso):
        super().__init__(nome, idade)
        self.curso = curso

    def __str__(self):
        return f"{super().__str__()} - {self.curso}"
```

Coleções e laços em Python

```
lista = [1, 2, 3]
lista.append(4)
lista.pop(0)

for x in lista:
    print(x)

for i in range(len(lista)):
    print(i, lista[i])

dados = {"nome": "Ana", "idade": 20}
for chave, valor in dados.items():
    print(chave, valor)

while lista:
    lista.pop()
```

11. Tutoriais de Resolução de Problemas Comuns

Criar uma classe a partir de um enunciado

- Leia o enunciado uma vez sem codar. Identifique o objeto principal.
- Substantivos principais viram possíveis classes: Pessoa, Agenda, Elevador, Dado.
- Características viram atributos: nome, altura, andarAtual, capacidade.
- Verbos/ações viram métodos: entrar, sair, subir, rolar, calcularIdade.
- Valores exigidos ao criar o objeto viram parâmetros do construtor.
- Regras de negócio viram validações dentro dos métodos.

```
// Enunciado: "Crie uma classe Dado. Ao criar, pode especificar lados; se não, 6. Pode rolar e consultar
último lançamento."
class Dado {
    private int lados;
    private int lado;
    private Random random = new Random();
    public Dado() { this(6); }
    public Dado(int lados) { this.lados = lados; rolar(); }
    public int rolar() { lado = random.nextInt(lados) + 1; return lado; }
    public int getLado() { return lado; }
}

# Python equivalente
import random
class Dado:
    def __init__(self, lados=6):
        self.lados = lados
        self.rolar()
    def rolar(self):
        self.lado = random.randint(1, self.lados)
        return self.lado
    def get_lado(self):
        return self.lado
```

Implementar encapsulamento

- Use atributos privados quando o valor não deve ser alterado diretamente.
- Crie getter quando o enunciado pedir consultar.
- Crie setter quando o enunciado pedir alterar e houver validação possível.
- Não exponha lista interna se isso permitir quebrar regras: retorne cópia ou crie métodos de operação.

```
class Pessoa {
    private String nome;
    private double peso;
    public String getName() { return nome; }
    public void setName(String nome) { this.nome = nome; }
    public double getPeso() { return peso; }
    public void setPeso(double peso) { if (peso > 0) this.peso = peso; }
}

class Pessoa:
    def __init__(self, nome, peso):
        self.nome = nome
        self._peso = peso
    @property
    def peso(self):
        return self._peso
    @peso.setter
    def peso(self, valor):
        if valor > 0:
            self._peso = valor
```

Resolver questões com herança

- Pergunte: "classe filha é uma classe mãe?". Se sim, herança pode fazer sentido.
- Coloque na superclasse atributos e métodos comuns.
- Use super(...) para inicializar a parte herdada.
- Use @Override em Java para sobrescrever comportamento.
- Não use herança apenas para reutilizar código se a relação conceitual não for "é um".

Resolver questões com polimorfismo

- Crie um tipo comum: superclasse abstrata ou interface.
- Faça métodos receberem o tipo comum, não cada subtipo separadamente.
- Em Java, lembre: tipo declarado define o que posso chamar; objeto real define qual implementação roda.

- Em Python, escreva função usando os métodos necessários e deixe qualquer objeto compatível funcionar.

Interfaces e classes abstratas

Quando usar	Interface	Classe abstrata
Preciso de contrato comum sem estado obrigatório	Sim	Pode, mas interface é mais limpa
Preciso compartilhar atributos/código comum	Não ideal	Sim
Classe deve assumir vários papéis	Sim, Java permite várias interfaces	Java só permite uma superclasse
Quero impedir instanciar tipo genérico	Pode ser interface	Sim, abstract class

Resolver questões com listas de objetos

```
ArrayList<Pessoa> pessoas = new ArrayList<>();
pessoas.add(new Pessoa("Ana", 20, 1.65));
pessoas.add(new Pessoa("Bruno", 22, 1.80));

// percorrer
for (Pessoa p : pessoas) System.out.println(p.getNome());

// buscar por nome
Pessoa achada = null;
for (Pessoa p : pessoas) {
    if (p.getNome().equals("Ana")) {
        achada = p;
        break;
    }
}

// média de idade
int soma = 0;
for (Pessoa p : pessoas) soma += p.getIdade();
double media = (double) soma / pessoas.size();

// remover por condição
pessoas.removeIf(p -> p.getIdade() < 18);

pessoas = [Pessoa("Ana", 20, 1.65), Pessoa("Bruno", 22, 1.80)]

# percorrer
for p in pessoas:
    print(p.nome)

# buscar por nome
achada = None
for p in pessoas:
    if p.nome == "Ana":
        achada = p
        break

# média de idade
media = sum(p.idade for p in pessoas) / len(pessoas)

# filtrar/remover por condição
pessoas = [p for p in pessoas if p.idade >= 18]
```

Resolver questões com menus

Menu textual quase sempre tem a mesma estrutura: inicializa dados, repete enquanto opção não é sair, lê opção, executa switch/if, volta ao menu.

```
import java.util.*;

public class MenuExemplo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<String> nomes = new ArrayList<>();
        int op;
        do {
            System.out.println("1-Inserir 2-Listar 3-Remover 0-Sair");
            op = Integer.parseInt(sc.nextLine());
            switch (op) {
                case 1:
                    System.out.print("Nome: ");
                    nomes.add(sc.nextLine());
                    break;
                case 2:
                    for (String n : nomes) System.out.println(n);
                    break;
                case 3:
                    System.out.print("Índice: ");
                    int i = Integer.parseInt(sc.nextLine());
                    if (i >= 0 && i < nomes.size()) nomes.remove(i);
                    break;
                case 0:
                    System.out.println("Fim");
                    break;
                default:
                    System.out.println("Opção inválida");
            }
        } while (op != 0);
        sc.close();
    }
}

def main():
    nomes = []
    while True:
        print("1-Inserir 2-Listar 3-Remover 0-Sair")
        op = input("Opção: ")
```

```

if op == "1":
    nomes.append(input("Nome: "))
elif op == "2":
    for n in nomes:
        print(n)
elif op == "3":
    i = int(input("Índice: "))
    if 0 <= i < len(nomes):
        nomes.pop(i)
elif op == "0":
    print("Fim")
    break
else:
    print("Opção inválida")

if __name__ == "__main__":
    main()

```

Resolver questões de modelagem

Passo	O que fazer	Exemplo rápido
1	Ler o enunciado todo.	Classe Elevador.
2	Marcar substantivos.	Elevador, prédio, pessoas.
3	Separar classes de atributos.	Elevador é classe; andarAtual e capacidade são atributos.
4	Marcar verbos.	subir, descer, entrar, sair.
5	Identificar relações.	ControleRemoto tem uma Televisao; Gato é Animal.
6	Codar construtor e métodos.	Inicializa estado válido e valida limites.
7	Testar mentalmente.	Não subir acima do último andar; não entrar acima da capacidade.

12. Soluções dos Problemas da Imagem Anexada

Enunciado resumido e ideia

- 1) Criar classe Par que representa dois dados quaisquer. Os tipos podem ser diferentes e são definidos na instanciação. Implementar getters.
- 2) Criar ListaLigada genérica. Cada item fica dentro de um NoLista genérico. Operações mínimas: inserir no início, inserir no fim, remover primeiro, remover último e toString.
- 3) Testar a lista com elementos do tipo Par<Integer, Double> em Java e Par<int, float> em Python.

Obs.: Conceitos usados: genéricos, encapsulamento, classe nó, referência para próximo nó, lista simplesmente ligada, toString/__str__, composição entre ListaLigada e NoLista.

Modelagem das classes

Classe	Responsabilidade	Atributos principais
Par<A,B>	Guardar dois valores de tipos possivelmente diferentes.	primeiro: A; segundo: B
NoLista<T>	Guardar um valor e referência para o próximo nó.	valor: T; proximo: NoLista<T>
ListaLigada<T>	Gerenciar a sequência de nós.	início, fim, tamanho

Solução completa em Java

```

// Salve como Main.java
class Par<A, B> {
    private A primeiro;
    private B segundo;

    public Par(A primeiro, B segundo) {
        this.primeiro = primeiro;
        this.segundo = segundo;
    }

    public A getPrimeiro() { return primeiro; }
    public B getSegundo() { return segundo; }

    @Override
    public String toString() {
        return "(" + primeiro + ", " + segundo + ")";
    }
}

class NoLista<T> {
    private T valor;
    private NoLista<T> proximo;

    public NoLista(T valor) {
        this.valor = valor;
        this.proximo = null;
    }

    public T getValor() { return valor; }
    public NoLista<T> getProximo() { return proximo; }
    public void setProximo(NoLista<T> proximo) { this.proximo = proximo; }
}

class ListaLigada<T> {
    private NoLista<T> inicio;
    private NoLista<T> fim;
    private int tamanho;

    public ListaLigada() {
        inicio = null;
        fim = null;
    }
}

```

```

    tamanho = 0;
}

public boolean estaVazia() { return tamanho == 0; }
public int getTamanho() { return tamanho; }

public void inserirInicio(T valor) {
    NoLista<T> novo = new NoLista<>(valor);
    if (estaVazia()) {
        inicio = novo;
        fim = novo;
    } else {
        novo.setProximo(inicio);
        inicio = novo;
    }
    tamanho++;
}

public void inserirFim(T valor) {
    NoLista<T> novo = new NoLista<>(valor);
    if (estaVazia()) {
        inicio = novo;
        fim = novo;
    } else {
        fim.setProximo(novo);
        fim = novo;
    }
    tamanho++;
}

public T removerPrimeiro() {
    if (estaVazia()) return null;
    T valor = inicio.getValor();
    inicio = inicio.getProximo();
    tamanho--;
    if (estaVazia()) fim = null;
    return valor;
}

public T removerUltimo() {
    if (estaVazia()) return null;
    T valor = fim.getValor();
    if (inicio == fim) {
        inicio = null;
        fim = null;
    } else {
        NoLista<T> atual = inicio;
        while (atual.getProximo() != fim) {
            atual = atual.getProximo();
        }
        atual.setProximo(null);
        fim = atual;
    }
    tamanho--;
    return valor;
}

@Override
public String toString() {
    StringBuilder sb = new StringBuilder("[");
    NoLista<T> atual = inicio;
    while (atual != null) {
        sb.append(atual.getValor());
        if (atual.getProximo() != null) sb.append(" -> ");
        atual = atual.getProximo();
    }
    sb.append("]");
    return sb.toString();
}

}

public class Main {
    public static void main(String[] args) {
        ListaLigada<Par<Integer, Double>> lista = new ListaLigada<>();
        lista.inserirInicio(new Par<>(1, 1.5));
        lista.inserirFim(new Par<>(2, 2.5));
        lista.inserirInicio(new Par<>(0, 0.5));

        System.out.println("Lista inicial: " + lista);
        System.out.println("Remove primeiro: " + lista.removerPrimeiro());
        System.out.println("Depois: " + lista);
        System.out.println("Remove último: " + lista.removerUltimo());
        System.out.println("Depois: " + lista);
    }
}

```

Solução completa em Python

```
from typing import Generic, TypeVar, Optional

A = TypeVar("A")
B = TypeVar("B")
T = TypeVar("T")

class Par(Generic[A, B]):
    def __init__(self, primeiro: A, segundo: B):
        self._primeiro = primeiro
        self._segundo = segundo

    def get_primeiro(self) -> A:
        return self._primeiro

    def get_segundo(self) -> B:
        return self._segundo

    def __str__(self) -> str:
        return f"({self._primeiro}, {self._segundo})"

class NoLista(Generic[T]):
    def __init__(self, valor: T):
        self.valor = valor
        self.proximo: Optional[NoLista[T]] = None

class ListaLigada(Generic[T]):
    def __init__(self):
        self.inicio: Optional[NoLista[T]] = None
        self.fim: Optional[NoLista[T]] = None
        self.tamanho = 0

    def esta_vazia(self) -> bool:
        return self.tamanho == 0

    def inserir_inicio(self, valor: T) -> None:
        novo = NoLista(valor)
        if self.esta_vazia():
            self.inicio = novo
            self.fim = novo
        else:
            novo.proximo = self.inicio
            self.inicio = novo
        self.tamanho += 1

    def inserir_fim(self, valor: T) -> None:
        novo = NoLista(valor)
        if self.esta_vazia():
            self.inicio = novo
            self.fim = novo
        else:
            assert self.fim is not None
            self.fim.proximo = novo
            self.fim = novo
        self.tamanho += 1

    def remover_primeiro(self) -> Optional[T]:
        if self.esta_vazia():
            return None
        assert self.inicio is not None
        valor = self.inicio.valor
        self.inicio = self.inicio.proximo
        self.tamanho -= 1
        if self.esta_vazia():
            self.fim = None
        return valor

    def remover_ultimo(self) -> Optional[T]:
        if self.esta_vazia():
            return None
        assert self.inicio is not None and self.fim is not None
        valor = self.fim.valor
        if self.inicio is self.fim:
            self.inicio = None
            self.fim = None
        else:
            atual = self.inicio
            while atual.proximo is not self.fim:
                assert atual.proximo is not None
                atual = atual.proximo
            atual.proximo = None
            self.fim = atual
        self.tamanho -= 1
        return valor

    def __str__(self) -> str:
        valores = []
        atual = self.inicio
        while atual is not None:
            valores.append(str(atual.valor))
            atual = atual.proximo
        return "[" + " ".join(valores) + "]"

if __name__ == "__main__":
    lista: ListaLigada[Par[int, float]] = ListaLigada()
    lista.inserir_inicio(Par(1, 1.5))
    lista.inserir_fim(Par(2, 2.5))

    lista.inserir_inicio(Par(0, 0.5))

    print("Lista inicial:", lista)
```

```
print("Remove primeiro:", lista.remover_primeiro())
print("Depois:", lista)
print("Remove último:", lista.remover_ultimo())
print("Depois:", lista)
```

Pontos importantes da solução

- A lista é genérica: `ListaLigada<T>` não sabe nem precisa saber qual é o tipo dos elementos.
- O nó também é genérico para guardar um valor `T` sem perder tipo.
- Manter uma referência fim facilita inserir no final em $O(1)$. Remover o último em lista simplesmente ligada ainda exige percorrer até o penúltimo, $O(n)$.
- Quando a lista fica vazia, início e fim devem virar `null/None`.
- `toString` percorre os nós do início ao fim e monta a representação textual.
- Par usa dois parâmetros de tipo, `A` e `B`, porque o primeiro e o segundo valor podem ter tipos diferentes.

13. Comparação Java vs Python

Tabela comparativa

Conceito	Java	Python
Classe	<code>public class Nome { ... }</code>	<code>class Nome: ...</code>
Arquivo	Classe <code>public</code> deve ficar em <code>Nome.java</code> .	Arquivo <code>.py</code> pode ter várias classes e não precisa ter mesmo nome.
Execução	<code>main: public static void main(String[] args).</code>	Comandos no topo executam; use <code>if __name__ == "__main__":</code> .
Construtor	<code>NomeDaClasse(...)</code> , sem retorno.	<code>__init__(self, ...)</code> .
Atributo	Declarado com tipo: <code>private int x;</code>	Criado por atribuição: <code>self.x = 0.</code>
Encapsulamento	<code>private/protected/public</code> ; getters/setters.	Convenções <code>_x</code> , <code>__x</code> ; properties.
Herança	<code>class Filho extends Pai.</code>	<code>class Filho(Pai):</code>
Chamar pai	<code>super(...)</code> , <code>super.metodo()</code> .	<code>super().__init__(...)</code> , <code>super().metodo()</code> .
Interface	<code>interface I; class C implements I.</code>	Protocol/ABC ou duck typing.
Classe abstrata	<code>abstract class; abstract method.</code>	ABC e <code>@abstractmethod</code> .
Polimorfismo	Tipo declarado + despacho dinâmico.	Duck typing + despacho dinâmico.
Sobrecarga	Mesmo nome, parâmetros diferentes.	Não há sobrecarga real; use parâmetros padrão.
Tipagem	Estática: compilador verifica tipos.	Dinâmica: valores têm tipos em execução.
Lista	<code>ArrayList<T>.</code>	<code>list.</code>
Mapa/dicionário	<code>HashMap<K,V>.</code>	<code>dict.</code>
Entrada	<code>Scanner/BufferedReader.</code>	<code>input().</code>
Saída	<code>System.out.println().</code>	<code>print().</code>
Exceções	<code>try/catch/finally.</code>	<code>try/except/finally.</code>

14. Checklist de Prova

Como começar qualquer questão de POO

- Escreva primeiro a modelagem: classes, atributos, métodos e relações.
- Defina construtores antes dos métodos complexos.
- Use `private` nos atributos em Java e getters/setters quando necessário.
- Implemente `toString/__str__` para facilitar testes e impressão.
- Teste com 2 ou 3 objetos no `main/bloco` de teste.
- Se houver coleção, teste lista vazia, um elemento e vários elementos.
- Se houver remoção, ajuste corretamente início/fim/tamanho.
- Se houver herança, coloque o que é comum na superclasse e o que é específico na subclasse.
- Se houver polimorfismo, métodos devem receber o tipo mais geral possível.
- Se houver interface, implemente todos os métodos exigidos.

Erros comuns em Java

- Esquecer que a classe `public` precisa ter mesmo nome do arquivo.
- Comparar `String` com `==` em vez de `equals`.
- Esquecer `static` no `main`.
- Esquecer `import java.util.*` para `Scanner/ArrayList/HashMap`.
- Chamar método exclusivo da subclasse usando referência da superclasse.
- Esquecer `@Override` ou errar assinatura do método sobrescrito.
- Não tratar casting quando há perda de informação.
- Remover de lista ligada sem atualizar fim quando a lista fica vazia.

Erros comuns em Python

- Errar indentação.
- Esquecer `self` no primeiro parâmetro dos métodos.
- Definir dois `__init__`; o segundo substitui o primeiro.
- Confundir lista mutável com tupla imutável.
- Achar que type hints impedem erro em execução; eles ajudam, mas não tornam Python estaticamente tipado.

- Não proteger código de teste com `if __name__ == "__main__":` quando o arquivo será importado.
- Alterar lista enquanto percorre sem cuidado.

Como explicar conceitos teóricos

Conceito	Resposta curta de prova
Classe	Molde que define atributos e métodos de objetos.
Objeto	Instância concreta de uma classe, com estado próprio.
Encapsulamento	Ocultar estado interno e controlar acesso por métodos.
Herança	Subclasse reaproveita/estende atributos e métodos da superclasse.
Polimorfismo	Mesmo tipo abstrato referencia objetos concretos diferentes e executa comportamento específico.
Interface	Contrato de métodos que uma classe se compromete a implementar.
Classe abstrata	Classe incompleta, não instanciável, usada como base comum.
Genérico	Classe/método parametrizado por tipo, definido na instanciação/uso.
Composição	Objeto complexo formado por objetos menores.

15. Fontes usadas

- 02-Uma classe.pdf: classe Java, modificadores de acesso, tipos, casting, controle, exceções, arrays e strings.
- Apresentação.pdf: paradigmas de programação, objetivos, conteúdo e metodologia da disciplina.
- Classesobjetos.pdf: fundamentos de POO, cadeira, construtores, composição, estáticos e exercícios de modelagem.
- HerancaJavaPython.pdf: herança, superclasse/subclasse, super, classes abstratas, Python, herança múltipla e MRO.
- Polimorfismo.pdf: polimorfismo, duck typing, sobrecarga e parâmetros opcionais em Python.
- Interfaces.pdf: interfaces, Comparable, genéricos e enunciado da lista ligada genérica.
- Interpretador Python - elementos básicos.pdf: interpretador, números, operadores, variáveis, strings, listas, matrizes e tuplas.
- Python - comandos e classes.pdf: script, E/S, arquivos, blocos, match, exceções, funções, passagem de parâmetros, classes e imports.
- Bozo (1).pdf: regras, classes Dado, RolaDados, Placar e Bozo.